

Advanced Web Programming

SIB245007 | D-IV Sistem Informasi Bisnis

Meeting 10: **Designing and Building a REST API**

Consistent endpoints, token authentication, and documentation you can trust

What You'll Learn

1. Design REST API endpoints that consistently follow HTTP method semantics, and curate the response shape with an API Resource
2. Implement token-based authentication with Sanctum for clients that don't have a cookie jar, including how that client itself consumes a protected endpoint
3. Document an API contract using the OpenAPI specification, so other teams don't need to read controller code to know how to call it

This deck covers concepts. Applying REST endpoints, Sanctum, and OpenAPI to Simple POS happens in the practicum jobsheet.

Part 1

Designing REST API Endpoints

Key Terms (1/2)

REST (Representational State Transfer): an API architectural style that maps operations to resources via standard HTTP methods (GET, POST, PUT, DELETE) and a URL that represents the resource itself.

Endpoint: one combination of an HTTP method and a URL path that performs one specific operation, for example `POST /api/login` .

Token: a random string representing a user's identity after a successful login, sent again with every subsequent request as proof that the user is already authenticated.

Key Terms (2/2)

Sanctum: Laravel's built-in authentication package for token-based APIs, designed for both SPA clients and pure mobile clients.

API Resource: a Laravel transformation class that turns an Eloquent model into a consistent JSON structure, separating the API response shape from the raw table column structure.

REST isn't a strict formal protocol, it's a set of shared conventions that make an API's behavior predictable just from its method and URL, without reading a line of documentation first.

HTTP Method Semantics Aren't Just Formality

GET must be safe and idempotent

- Never changes state on the server
- Calling it repeatedly = same effect as once
- Clients can safely retry without worry

POST may change state

- Can create a new row in the database
- Calling it twice can create two rows
- The client must handle that risk itself

`GET /api/products` called repeatedly because of a dropped connection never duplicates data. `POST /api/transactions` called twice because the first request timed out genuinely can create two transactions, not something safe to assume won't happen.

API Resource: Curating the Response Shape

An Eloquent model returned directly as JSON can leak internal columns (`updated_at` , `password`) or relationships the client has no use for. An API Resource ensures every endpoint returns a curated, consistent JSON shape.

```
class ArticleResource extends JsonResource
{
    public function toArray($request): array
    {
        return [
            'id' => $this->id,
            'title' => $this->title,
            'author' => $this->author->name,
        ];
    }
}
```

Only three fields are chosen to appear in the JSON, so whatever column gets added to the `articles` table later doesn't automatically leak to the client.

Part 2

Token Authentication with Sanctum

Cookie Sessions vs Tokens: Different Clients, Different Mechanisms

Web pages (Meeting 7)

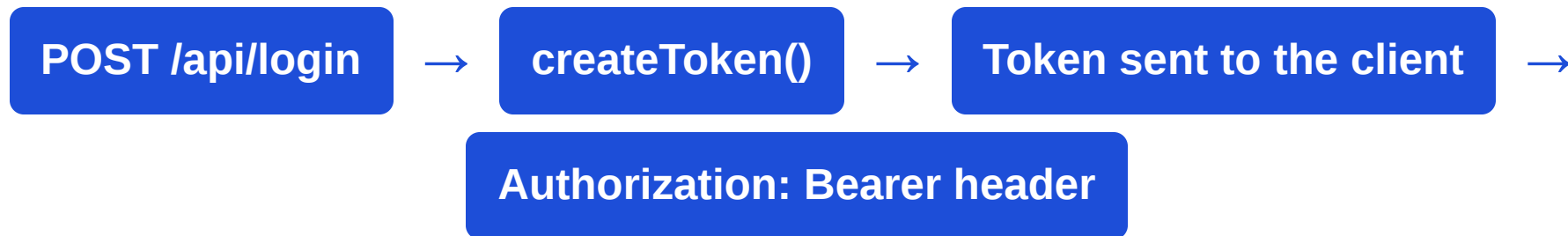
- The browser stores cookies automatically
- Sessions are matched via that cookie
- Suits clients that have a cookie jar

API clients (mobile, external scripts)

- Have no cookie jar like a browser does
- The client stores the token itself
- Sends it explicitly with every request

Both prove "who is making this request", only the mechanism for storing and sending that proof differs, matching the kind of client using it.

The Sanctum Authentication Flow



```
$user = Auth::user();  
$token = $user->createToken('mobile-client')->plainTextToken;
```

`createToken()` stores that token as a hash in the `personal_access_tokens` table, and `->plainTextToken` is the only chance that raw token is ever visible. After this login response, the server can never show that same token in readable form again.

Protecting a Route with auth:sanctum

A route that needs protection just gets wrapped in the `auth:sanctum` middleware, similar to the `role:admin` pattern from Meeting 7, except what's checked this time is the presence and validity of a token via the `Authorization` header, not a cookie session.

```
Route::middleware('auth:sanctum')->group(function () {  
    Route::get('/articles', [ArticleController::class, 'index']);  
    Route::post('/articles', [ArticleController::class, 'store']);  
});
```

Without a valid `Authorization: Bearer <token>` header, this middleware rejects the request before it ever reaches the controller, exactly like `role:admin` rejects a non-admin user before reaching the protected action.

Part 3

Consuming and Documenting the API

A Client Must Check Status Before Reading Content

An API client (a mobile app, a PHP script, or any language) must never assume every request succeeds.

```
$login = Http::post("{baseUrl}/api/login", [  
    'email' => $email,  
    'password' => $password,  
]);  
  
if ($login->failed()) {  
    // handle the error here, don't proceed to the next step  
}  
  
$token = $login->json('token');
```

A client that reads `$login->json('token')` directly without checking `failed()` first fails silently: wrong credentials make `$token` end up `null` with no clear reason why, and the error only explodes a few steps later somewhere confusing.

Distinguishing Error Types

Not every error is handled the same way. The HTTP status code tells the client what the right next move is.

```
match ($response->status()) {  
    201 => /* saved, read $response->json('data') */,  
    401 => /* invalid token, the client must log in again */,  
    422 => /* input rejected, fix the input, not re-login */,  
    default => /* some other error, show it to the user */,  
};
```

401 means the problem is the client's identity (an expired or revoked token), while 422 means the problem is the data that was sent. Treating both as "just log in again" makes the client retry login pointlessly when the real problem is the input.

OpenAPI: A Contract, Not a Note That Goes Stale

OpenAPI: a structured specification (YAML or JSON) that describes an API's endpoints, readable by other tools such as interactive documentation generators or client code generators.

```
paths:
  /api/articles:
    get:
      summary: Get the list of articles
      responses:
        '200':
          description: List of articles
```

Plain text notes in a README go stale fast, because nothing forces anyone to update them when a response field gets renamed. A structured format like OpenAPI can be checked by automated tooling, so stale documentation gets caught by a failing test, not by another team's complaint later.

Applying This to Simple POS

You'll design and build `POST /api/login`, `GET /api/products`, and `POST /api/transactions` for Simple POS in the practicum jobsheet: protected by a Sanctum token, consumed by a real HTTP client, and documented with OpenAPI.

Full code: github.com/se-polinema/simple-pos-ch09

Summary (1/2)

- REST endpoints consistently follow HTTP method semantics: `GET` is safe and idempotent, `POST` may change state and risks duplication if retried
- An API Resource curates the JSON shape returned, separating the API response from the raw table column structure
- Sanctum issues a token via `createToken()`, sent by the client through the `Authorization: Bearer` header, checked by the `auth:sanctum` middleware on protected routes

Summary (2/2)

- An API client must check `failed()` before reading a response's content, and distinguish handling 401 (log in again) from 422 (fix the input), instead of treating every error the same way
- An OpenAPI specification documents each endpoint's `paths`, `requestBody`, and `responses` in a structured format that automated tooling can check, not just a passive note that goes stale

References & Discussion

Official Laravel Sanctum documentation, the OpenAPI specification, and Fielding's paper on REST architectural principles

Full code: `github.com/se-polinema/simple-pos-ch09`

Next meeting: the start of the PBL (Project Based Learning) project, you'll plan the features and architecture of your own web project